

Passo 1: Configurando o Ambiente

Instale as dependências necessárias:

bash

Copiar código

```
npm install --save-dev jest @testing-library/react-native
```

1.

Configure o Jest: Certifique-se de que o Jest está configurado no seu projeto. No

`package.json`, adicione:

json

Copiar código

```
"scripts": {  
  "test": "jest"  
},  
"jest": {  
  "preset": "react-native",  
  "setupFilesAfterEnv":  
    ["@testing-library/jest-native/extend-expect"]  
}
```

2.

Passo 2: Escrevendo um Teste Simples

Aqui está um exemplo de um teste simples para um componente React Native.

Componente: `Button.js`

javascript

Copiar código

```
import React from 'react';  
import { Button, Text } from 'react-native';  
  
const MyButton = ({ title, onPress }) => (  
  <Button title={title} onPress={onPress} />  
);  
  
export default MyButton;
```

Teste: `Button.test.js`

javascript

Copiar código

```
import React from 'react';
import { render, fireEvent } from '@testing-library/react-native';
import MyButton from './Button';

describe('MyButton', () => {
  it('renders correctly with the given title', () => {
    const { getByText } = render(<MyButton title="Press me"
onPress={() => {}} />);
    expect(getByText('Press me')).toBeTruthy();
  });

  it('calls the onPress handler when pressed', () => {
    const onPressMock = jest.fn();
    const { getByText } = render(<MyButton title="Press me"
onPress={onPressMock} />);

    fireEvent.press(getByText('Press me'));
    expect(onPressMock).toHaveBeenCalledTimes(1);
  });
});
```

Passo 3: Executando os Testes

Rode o comando para executar os testes: `npm test`

Explicação

- **render:** Monta o componente e retorna métodos para interagir com ele no teste.
- **fireEvent:** Simula eventos como cliques, toques, etc.
- **expect:** Verifica se a saída do componente corresponde ao esperado.

Exemplo de Componente

Vamos supor que você tenha um componente que cria um contato ao clicar em um botão. Esse componente pode ser algo como:

```
javascript
Copiar código
// ContactButton.js
import React from 'react';
import { Button } from 'react-native';
```

```
const ContactButton = ({ createContact }) => {
  return (
    <Button
      title="Create Contact"
      onPress={() => {
        createContact({ name: 'John Doe', phone: '123-456-7890' });
      }}
    />
  );
};

export default ContactButton;
```

Teste Unitário

Agora, vamos escrever um teste que verifica se o método `createContact` foi chamado com os dados corretos quando o botão é pressionado.

javascript

Copiar código

```
// ContactButton.test.js
import React from 'react';
import { render, fireEvent } from '@testing-library/react-native';
import ContactButton from './ContactButton';

describe('ContactButton', () => {
  it('calls createContact with correct data when button is pressed',
    () => {
      const createContactMock = jest.fn();
      const { getByText } = render(<ContactButton
        createContact={createContactMock} />);

      // Simula o clique no botão
      fireEvent.press(getByText('Create Contact'));

      // Verifica se createContact foi chamado com os dados corretos
      expect(createContactMock).toHaveBeenCalledWith({
        name: 'John Doe',
        phone: '123-456-7890',
      });
    });
});
```

```
    // Verifica se createContact foi chamado apenas uma vez
    expect(createContactMock).toHaveBeenCalledTimes(1);
  });
});
```

Explicação

- **createContactMock**: É um mock function que simula a função `createContact`. O Jest permite rastrear chamadas a essa função para garantir que ela foi chamada corretamente.
- **fireEvent.press**: Simula o evento de pressionar o botão.
- **expect(createContactMock).toHaveBeenCalledTimes**: Verifica se a função mock foi chamada com os parâmetros esperados.

Executando o Teste

Para executar o teste, use o comando: `npm test`

Testando chamadas de API

Quando a função `createContact` faz uma chamada a uma API, você pode testar o comportamento usando mocks para a função de API. O Jest permite que você faça isso facilmente com `jest.mock()` para interceptar a chamada à API e simular diferentes respostas.

Vamos assumir que você tem uma função `saveContact` que faz uma chamada à API para salvar o contato e que o componente chama essa função quando o botão é pressionado.

Função de API (`api.js`):

javascript

Copiar código

```
export const saveContact = async (contact) => {

  const response = await fetch('https://api.example.com/contacts', {

    method: 'POST',

    headers: {

      'Content-Type': 'application/json',
```

```
    },  
    body: JSON.stringify(contact),  
  });  
  
  if (!response.ok) {  
    throw new Error('Failed to save contact');  
  }  
  
  return await response.json();  
};
```

Componente (**ContactButton.js**):

javascript

Copiar código

```
import React from 'react';  
  
import { Button } from 'react-native';  
  
import { saveContact } from '../api';  
  
const ContactButton = () => {  
  const handlePress = async () => {  
    try {  
      const contact = { name: 'John Doe', phone: '123-456-7890' };  
  
      await saveContact(contact);  
  
      alert('Contact saved successfully!');  
    } catch (error) {
```

```
        alert('Failed to save contact');
    }

};

return <Button title="Create Contact" onPress={handlePress} />;
};

export default ContactButton;
```

Testando o Componente com Chamada à API

Agora, você precisa mockar a função `saveContact` para simular a chamada à API no teste.

javascript

Copiar código

```
// ContactButton.test.js

import React from 'react';

import { render, fireEvent, waitFor } from
 '@testing-library/react-native';

import ContactButton from './ContactButton';

import { saveContact } from './api';

jest.mock('./api');

describe('ContactButton', () => {

    it('calls saveContact with correct data when button is pressed',
    async () => {
```

```
// Simula uma resposta de sucesso da API

saveContact.mockResolvedValueOnce({ id: 1, name: 'John Doe',
phone: '123-456-7890' });

const { getByText } = render(<ContactButton />);

fireEvent.press(getByText('Create Contact'));

// Espera que a função saveContact seja chamada com os dados
corretos

await waitFor(() => {

  expect(saveContact).toHaveBeenCalledWith({

    name: 'John Doe',

    phone: '123-456-7890',

  });

});

// Verifica se a função foi chamada apenas uma vez

expect(saveContact).toHaveBeenCalledTimes(1);

});

it('shows an alert on success', async () => {

  saveContact.mockResolvedValueOnce({ id: 1, name: 'John Doe',
phone: '123-456-7890' });

  const { getByText } = render(<ContactButton />);
```

```
fireEvent.press(getByText('Create Contact')));

await waitFor(() => {
  expect(getByText('Contact saved successfully!')).toBeTruthy();
});

});

it('shows an alert on failure', async () => {
  saveContact.mockRejectedValueOnce(new Error('Failed to save
contact'));

  const { getByText } = render(<ContactButton />);

  fireEvent.press(getByText('Create Contact'));

  await waitFor(() => {
    expect(getByText('Failed to save contact')).toBeTruthy();
  });

});

});
```

Explicação

- **jest.mock('./api')**: Intercepta todas as importações de **saveContact** para que você possa simular sua resposta durante o teste.

- **mockResolvedValueOnce**: Simula uma promessa resolvida (sucesso) quando `saveContact` é chamada.
- **mockRejectedValueOnce**: Simula uma promessa rejeitada (erro) quando `saveContact` é chamada.
- **waitFor**: Como `saveContact` é assíncrono, `waitFor` garante que o código espere a conclusão da operação antes de fazer as verificações.
- **expect(getByText(...))**: Verifica se o texto correspondente ao sucesso ou falha aparece na tela após a execução da operação.

Executando o Teste

Execute o comando `npm test` para rodar o teste.